



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2023/2024

Modelação e Simulação de Sistemas Naturais
Relatório: Projeto Final

Turma: 31D

Grupo: 30

Nome: Miguel Alcobia

Número: 50746

Nome: Fábio Pestana

Número: 50756

Professor: Paulo Vieira

Data: 20 de Janeiro de 2024

Índice

Lista de Figuras	3
Introdução	4
História e Simulações	5
Contexto da História	5
Simulação no <i>Silico</i>	6
Leitura dos resultados	7
Implementação	8
Geração do Terreno	8
Diagrama UML	10
Descrição da Classe Mover	11
Descrição da Classe Body	11
Descrição da Classe ParticleSystem	11
Descrição da Classe HunterParticleSystem	11
Descrição da Classe DNA	12
Descrição da Classe Eye	12
Descrição da Classe Behavior	12
Descrição da Classe Wander	12
Descrição da Classe Pursuit	12
Descrição da Classe Seek	12
Descrição da Classe Evade	13
Descrição da Classe AvoidObstacles	13
Descrição da Classe CellularAutomata e MajorityCA	13
Descrição da Classe Cell e MajorityCell	13
Descrição da Classe Patch	13
Descrição da Classe Terrain	13
Descrição da Classe Animal	13
Descrição da Classe Predator e Prey	13
Descrição da Classe Hunter	14
Descrição da Classe Player	14
Descrição da Classe WorldConstants	14
Descrição da Classe Ecosystem	14
Descrição da Classe GameApp	14
Conclusões	16
Bibliografia e Ferramentas	17

Lista de Figuras

Figura 1	- Sprites do terreno	5
Figura 2	- Sprites dos personagens	5
Figura 3	- Diagrama de Stocks & Flows	7
Figura 4	- Gráfico de Stocks & Flows	7
Figura 5	- Ilustração dos comportamentos de Pursuit e Evade	8
Figura 6	- Sprites dos corações do Herói	9
Figura 7	- UML	10
Figura 8	- Diversos Ecrãs do jogo	15

Introdução

Neste projeto final, os alunos tinham que desenvolver um jogo/simulação, aplicando os conhecimentos adquiridos ao longo do semestre, tais como:

- “1. Modelação baseada em Stocks & Flows;*
- 2. Modelação baseada em agentes (e.g., Boids);*
- 3. Geração procedimental de conteúdos usando objetos fractais (e.g., árvores) e autómatos celulares (e.g., terrenos);*
- 4. Simulação do movimento de objetos usando as leis da física;*
- 5. Modelação de comportamentos individuais e de grupo em agentes autónomos;*
- 6. Simulação da Seleção Natural que permite explicar a evolução das características genéticas de uma população (brevemente);”*

No desenvolvimento do projeto, os alunos usaram a experiência das aulas de laboratório e dos Trabalhos Práticos, ou seja, utilizar as bibliotecas do Processing para desenvolver aplicações gráficas e interativas e ferramentas de simulação como o Silico.

Os alunos decidiram fazer um ecossistema com Leões e Zebras.

História e Simulações

Contexto da História

Como apresentando no final da Introdução, o ecossistema apresentará a interação de predadores e presas. Neste contexto as presas serão Zebras e os predadores serão Leões que coexistem numa savana africana. O terreno da savana será representado por um autômato celular 2D, que assumem um determinado tipo de terreno (Vazio, Obstáculo, Fértil, Alimento).

Os Leões terão como objetivo perseguir e comer as Zebras para alimentarem-se e ganharem energia para continuar a perseguição e para reproduzirem-se. As Zebras, por sua vez, têm de fugir dos Leões, comer erva, onde ganham energia para sobreviver e reproduzir.

Tanto as presas como os predadores, também têm de evitar obstáculos no terreno (Pedras), se não, estes são penalizados por isso (perdem energia). Isto acontece, pois a escalada das pedras, exige maior gasto energético.

Além dos Leões e Zebras temos um Herói (que é controlado pelo *Player*) e Caçadores. O papel do Herói é defender os animais na savana dos Caçadores. O Herói foi criado pelos animais e assumiu para si a responsabilidade de protegê-los. O Homem (caçador) como é o predador *Ápex* do sistema, é capaz de matar os animais no sistema, sendo no caso os Leões e as Zebras. Em caso de insucesso na caça, os Leões podem matar os Caçadores, enquanto as Zebras só se limitam a fugir em qualquer cenário.

Contudo, os Caçadores, a tentarem aumentar o resultado da sua caça, camuflam-se em arbustos e nas pedras do terreno. O terreno da savana é composto por Pedras, Terra (terreno infértil e fértil) e por Arbustos, que servem de alimento às Zebras.

Abaixo seguem os *sprites* do terreno:

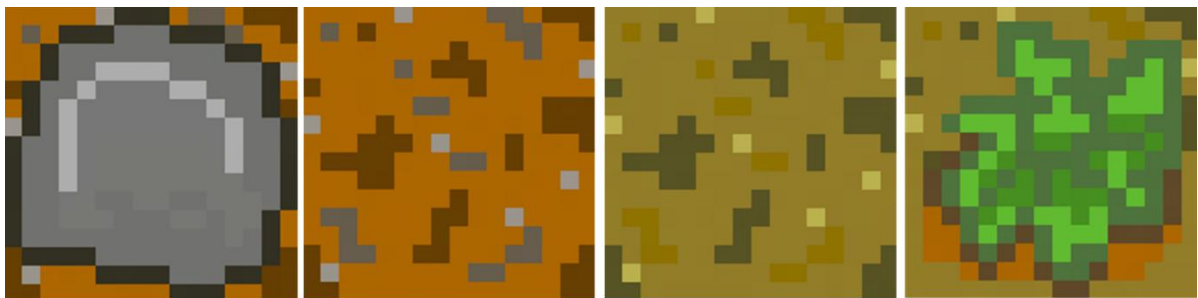


Figura 1 - Sprites do terreno

A seguir, seguem os *sprites* dos personagens (Leões (machos e fêmeas), Zebras, Caçadores (normal, camuflado nas pedras e num arbusto) e o Herói:



Figura 2 - Sprites dos personagens

Simulação no *Silico*

Link: <https://silico.app/@mikos2004/onboardingexample/app?path=183>

Com a ajuda da ferramenta Silico, tentou-se perceber o comportamento a esperar do ecossistema, ao longo do tempo.

Por estarmos a trabalhar com um ecossistema onde existe uma interação entre presas e predadores, as equações de Lotka-Volterra serão as ideais para ilustrar a evolução da população de Zebras e Leões ao longo do tempo.

Para as Zebras, que são as presas, temos a seguinte equação:

$$\frac{dz}{dt} = \alpha \times Zebras - \beta \times Zebras \times Leões$$

Para os Leões, que são os predadores, temos a seguinte equação:

$$\frac{dl}{dt} = \delta \times Leões \times Zebras - \gamma \times Leões$$

Começando por elaborar os Stocks & Flows das Zebras. Temos de ter em atenção de que a população não tem recursos ilimitados, o que significa que temos de controlar o crescimento da população. Como o terreno tem alimento limitado, é preciso restringir o crescimento da população através da capacidade de carga do sistema (o máximo de animais que o sistema suporta), como mostra a seguinte fórmula:

$$\frac{dz}{dt} = \alpha \times Zebras \left(1 - \frac{Zebras}{Capacidade\ de\ carga}\right)$$

Para calcular a capacidade de carga, começamos por ver o tamanho da grelha do autómato que simulará o terreno. Como temos 20 linhas e 20 colunas, concluímos que o terreno é composto por 400 células. Cada célula fértil demora, em média, 15 segundos até que volte a ter alimento disponível e cada uma delas dá 4 calorias. Sendo que 60% das células são alimento (aproximadamente 240 células), chegamos à conclusão de que o terreno dispõe de 960 calorias.

Com estes dados também conclui-se que temos uma produção de 64 calorias por segundo ($\frac{960}{15}$). Assumindo que cada zebra demora 1 segundo a alimentar-se temos uma capacidade igual a $\frac{64}{1} = 64$.

Para chegar à Taxa de Natalidade Máxima, recorremos à seguinte fórmula:

$$Z_{(t)} = Z_0 \times e^{\alpha t}$$

Presumindo que as Zebras demoram 5 segundos a reproduzirem-se a equação vai ganhando o seguinte aspeto:

$$2 \times Z_0 = Z_0 \times e^{5\alpha} \quad (=) \quad 2 = e^{5\alpha} \quad (=) \quad \ln(2) = 5\alpha \quad (=) \quad \frac{\ln(2)}{5} = \alpha = 0.14$$

Como observado na fórmula apresentada anteriormente, que representa a evolução da população dos Leões ao longo do tempo, percebemos que o número de nascimentos dos Leões depende do número de Leões e Zebras existentes no sistema, já que as Zebras são uma fonte de energia para os Leões, permitindo que estes se reproduzam. A taxa de natalidade dos Leões (δ) como é constante, definiu-se um valor de 0.01.

Com a interação entre as presas e os predadores, a população dos Leões irá variar ao longo do tempo, assim como a população de Zebras, que aumentará quanto menos predadores existirem e diminuirá quantos mais

predadores existirem.

Agora temos tudo o que é preciso para desenhar o diagrama de Stocks & Flows, que ficou da seguinte forma:

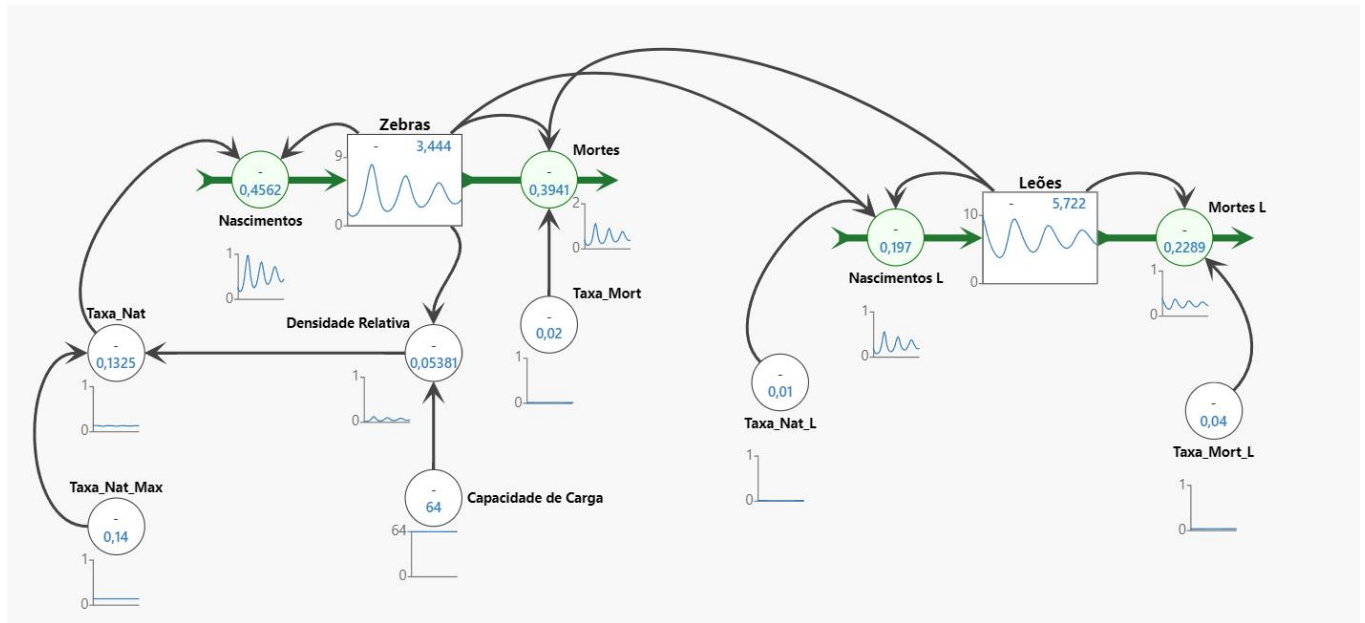


Figura 3 - Diagrama de Stocks & Flows

Leitura dos resultados

O diagrama acima deu origem ao seguinte gráfico:

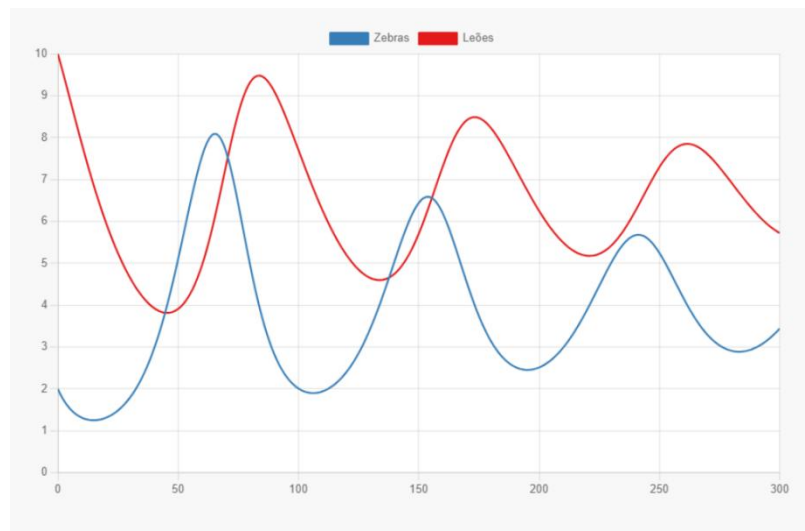


Figura 4 - Gráfico de Stocks & Flows

A população dos Leões começa com um valor inicial de 10 animais, enquanto o das Zebras começa com o valor inicial de 2 animais. Com a demora do crescimento da população das Zebras, os Leões começam a morrer, mas com o nascimento de mais presas, os Leões conseguem energia e alimento para sobreviverem e a sua população também volta a crescer.

Entretanto com a caça dos Leões, as Zebras começam a morrer em grande quantidade e a sua população diminui, por consequência o número de Leões também cai. Esta dinâmica vai se repetindo ao longo do tempo, mas nunca se verifica a extinção de uma das espécies.

Implementação

Geração do Terreno

O terreno é baseado num autômato celular 2D e é gerido na classe Terrain que deriva da classe MajorityCA, isto porque a regra da maioria é aplicada ao autômato para a criação dos biomas. Cada célula do autômato é um objeto da classe Patch que deriva da classe MajorityCells. Existem 4 tipos de Patches, cada um corresponde a um tipo de terreno e caso o terreno seja fértil, passa a ter alimento disponível se o tempo de regeneração já tiver passado.

Comportamento das Zebras

A Zebra é o animal com os comportamentos e mecânicas mais simples do ecossistema. Uma Zebra é um objeto da classe Prey, que tal como os restantes animais, tem como classe mãe a classe Animal. Se a Zebra andar sobre um terreno com erva, esta come-a e o terreno fica em estado fértil até regenerar o alimento.

Em termos de comportamentos as presas têm Wander, AvoidObstacle e Evade associados a elas. O comportamento Wander serve só para o animal vaguear pelo mapa; o AvoidObstacle faz com que o animal se desvie de um obstáculo, caso detete um; o Evade será ativado quando a presa avista o seu predador e, consequentemente, começa a fugir dele.

Comportamento dos Leões

Um Leão é um objeto da classe Predator. O objetivo do Leão é alimentar-se das Zebras para ganhar energia e consequentemente reproduzir-se, podendo também matar Caçadores.

Em termos de comportamentos os predadores têm Wander, AvoidObstacle e Pursuit associados a eles. O Pursuit será ativado quando a presa avista a sua presa e, consequentemente, começa persegui-la.

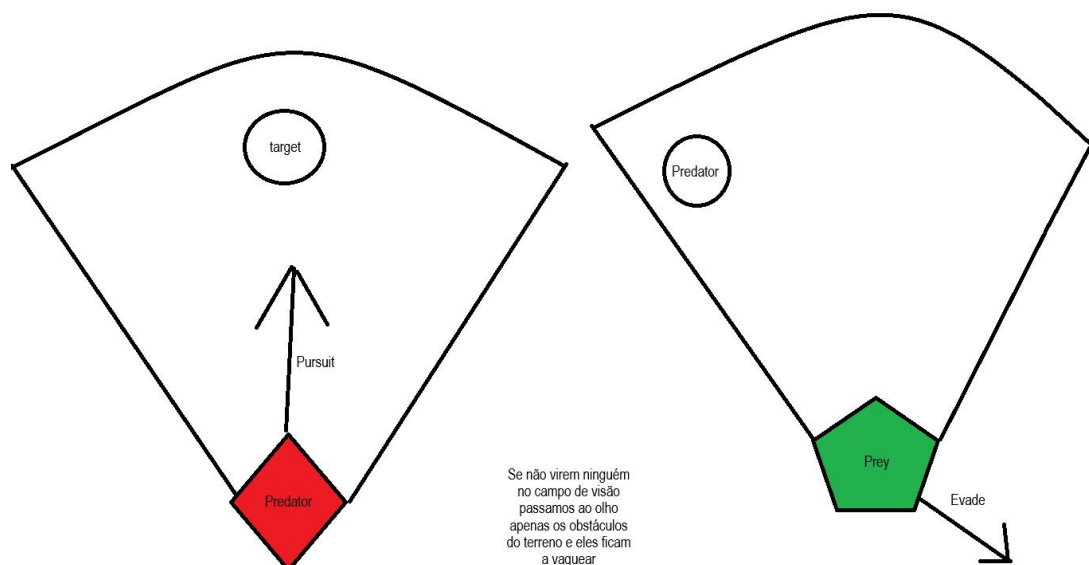


Figura 5 - Ilustração dos comportamentos de Pursuit e Evade

Comportamento do Caçador

O Caçador caça tanto as Zebras como os Leões, sendo que as primeiras morrem sem dar luta, mas os Leões podem matar o caçador. Caso não tenham um alvo em vista e estejam num terreno que seja de Alimento ou Pedra, ficam camuflados no respectivo ambiente.

Caso o Caçador não mate nenhum animal irá acabar por morrer, deixando um rasto (um sistema de partículas que fica ativo durante algum tempo).

No que diz respeito a comportamentos, o Caçador só lida com o Pursuit.

Comportamento do Herói

O Herói protege os animais e têm mecânicas próprias por ser o avatar do jogador. A sua energia é inicializada com um determinado valor e começa com 3 vidas. Para movimentar o Herói, o jogador deve clicar no destino pretendido, sendo que o Player fará Seek a essa posição. Para matar, o Player deve pressionar a tecla ESPAÇO.

Por cada morte, o jogador ganha energia; contudo, por cada animal morto, o Herói é penalizado com a perda de uma vida.

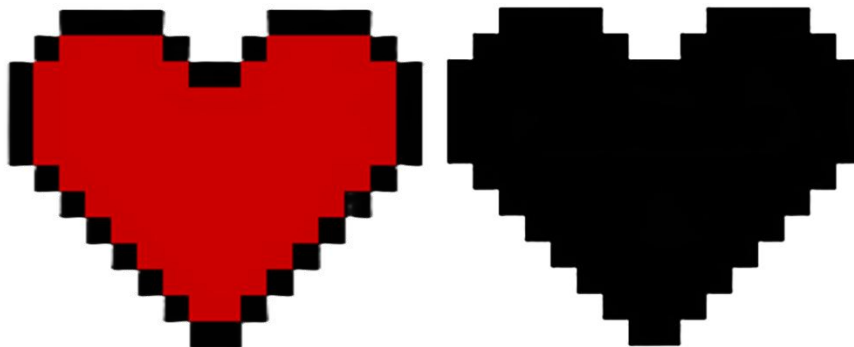


Figura 6 - Sprites dos corações do Herói

Diagrama UML

No final do projeto, produziu-se o diagrama de classes do trabalho e o resultado foi o seguinte:

(Devido à dimensão do UML, a resolução ficou má para a respetiva leitura, contudo o UML vai na pasta mod do trabalho)

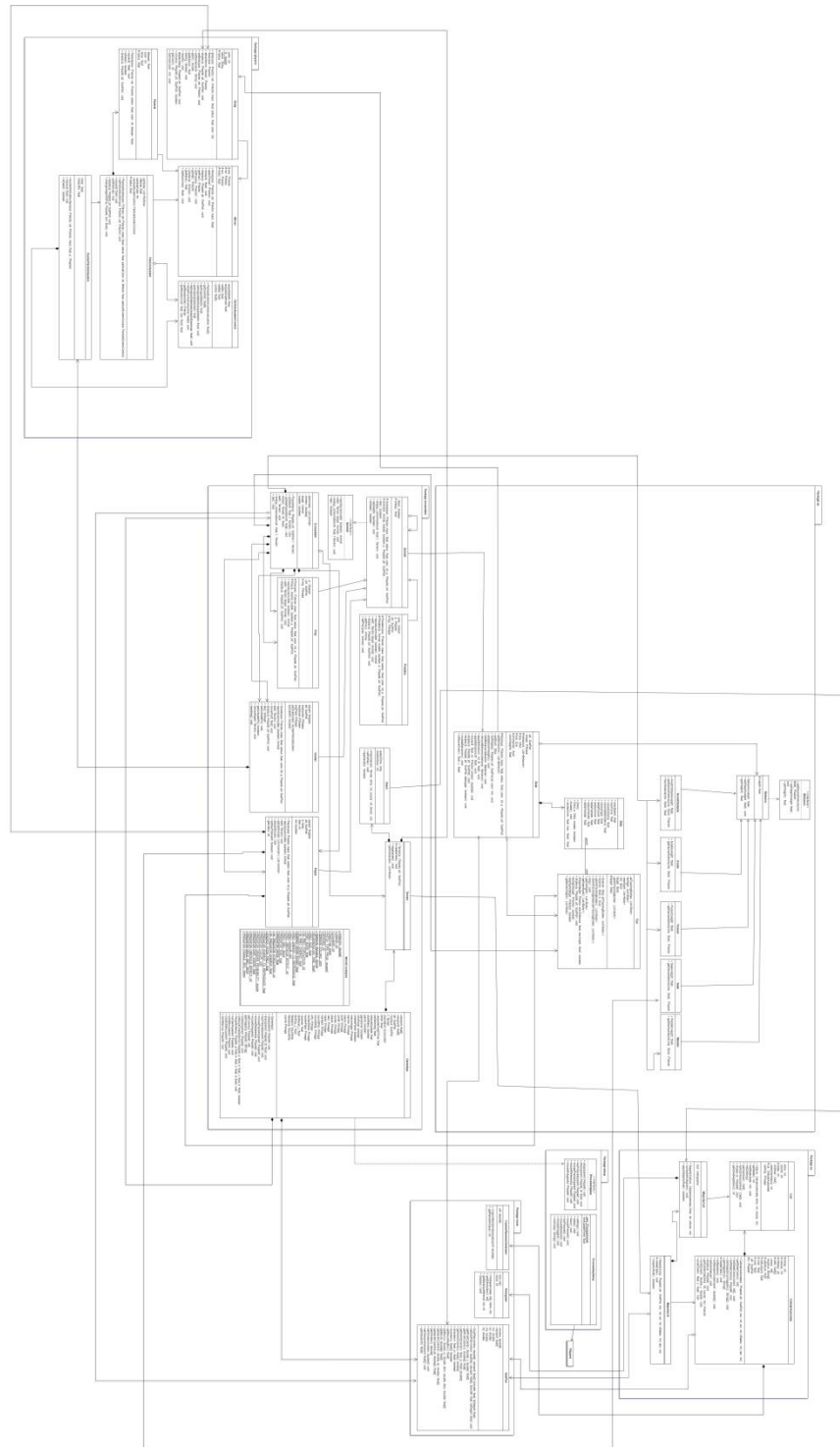


Figura 7 - UML

Descrição da Classe Mover

Esta classe pretende simular um corpo/objeto que se movimentará e tem como atributos: o vetor de posição (pos), um de velocidade (vel), um de aceleração (acc) e um valor float que diz respeito à massa (mass):

Esta classe tem dois métodos importantes:

- `applyForce(PVector f)`: Aplica uma força ao corpo. A força é dividida pela massa do corpo antes de ser adicionada à aceleração, segundo a segunda lei de Newton: $F = m \times a$
- `move(float dt)`: Atualiza a posição do corpo com base na velocidade e na aceleração, consoante um intervalo de tempo dt.

Descrição da Classe Body

A classe Body é a classe que dará origem à classe Boid, que por sua vez originará a class Animal. A classe Body herda os atributos e os métodos da classe Mover. Tem como atributos próprios a cor (int color), e a constante gravitacional $G = 6.67e-11$ (double G). Além dos gets e sets criou-se os seguintes métodos:

- `attraction(Mover m)`: Calcula e retorna o vetor de atração gravitacional entre o corpo e outro objeto Mover, que é passado como parâmetro.
- `resetG()`: Restaura a constante gravitacional para o valor padrão.

Descrição da Classe ParticleSystem

Esta classe pretende simular um sistema de partículas, que originará a classe HunterParticleSystem e tem como atributos: a lista de partículas do sistema (particles), um float com o tempo de vida da partícula (lifetime), um int para a sua cor (particleColor), um sistema de controlo do ParticleSystem (particleSystemControl) e um valor float que diz respeito ao raio da partícula (radius):

Esta classe tem os seguintes métodos:

- `move(float dt)`: Atualiza a posição das partículas e vai removendo as partículas já mortas.
- `addParticle()`: Cria uma nova partícula e adiciona-a à lista.

Descrição da Classe HunterParticleSystem

Esta classe deriva do ParticleSystem e simula um sistema de partículas, que entra em ação quando um Caçador morre e diferencia-se da sua classe mãe, porque se estabelece um timer de vida para o sistema. Tem como atributos um valor float que conta o tempo da partícula (timer) e um float com o tempo de vida do sistema de partículas (timeOfPs).

Quando o timer é maior que o timeOfPs, a função `isDead()` retorna true.

Descrição da Classe DNA

A classe DNA representa o DNA de uma criatura com as suas características próprias, apresentadas nos atributos da classe. Entre elas estão a Velocidade Máxima, a Força Máxima, Distância da Visão, Distância Segura da Visão, Ângulo da Visão e os parâmetros para o funcionamento dos comportamentos.

Descrição da Classe Eye

Esta classe pretende simular um olho/campo de visão de um Boid e tem como atributos: 4 Listas de Body: allTrackingBodies (todos os Bodies que o olho vê); farSight (os bodies que estão dentro do campo visão, mas ainda longe); nearSight (os que estão muito próximo do campo de visão) e obstaclesInSight (os obstáculos que o Boid vê). Além disso, tem o Boid me (o próprio Boid) e o target (Boid alvo).

Os métodos da classe são:

- look(): Método para detectar objetos dentro do campo de visão.
- getFarSight(): Obtém os Boids detetados dentro do campo de visão.
- getObstaclesInSight(): Obtém os Boids detetados dentro do campo de visão.
- getNearInSight(): Obtém os Boids detetados dentro do campo de visão mais próximo.
- inSight(): Verifica se um objeto está dentro do campo de visão.

Descrição da Classe Behavior

A classe Behavior dará origem a todos os comportamentos, tendo como único atributo um valor float que apresenta o peso desejado que o comportamento deverá ter (weight).

Descrição da Classe Wander

A classe Wander implementa o comportamento Wander, que consiste no vaguear do corpo pelo mapa.

Descrição da Classe Pursuit

A classe Pursuit implementa o comportamento Pursuit, que consiste na perseguição de um alvo em movimento, prevendo a sua futura posição.

Descrição da Classe Seek

A classe Seek implementa o comportamento Seek, que consiste na perseguição de um alvo “imóvel”, até chegar à sua posição.

Descrição da Classe Evade

A classe Evade implementa o comportamento Evade, que consiste na fuga de um alvo (neste caso o predador) em movimento, prevendo a sua futura posição.

Descrição da Classe AvoidObstacles

A classe AvoidObstacles implementa o comportamento AvoidObstacles, que consiste no desvio de obstáculos presentes no terreno, invertendo o sentido do deslocamento.

Descrição da Classe CellularAutomata e MajorityCA

A classe CellularAutomata implementa um autômato celular, enquanto a classe MajorityCA herda da CellularAutomata, mas implementa a regra da maioria.

Descrição da Classe Cell e MajorityCell

A classe Cell implementa um célula de um autômato celular, enquanto a classe MajorityCell herda da Cell, mas possui um histograma para ser possível implementar a regra da maioria.

Descrição da Classe Patch

A classe Patch é uma MajorityCell, com métodos para metê-la fértil e para regenerar o alimento.

Descrição da Classe Terrain

A classe Terrain é uma MajorityCA, com métodos para regenerar o alimento nas células apropriadas.

Descrição da Classe Animal

A classe Animal é um Boid, com métodos para consumir e obter energia e morrer.

Descrição da Classe Predator e Prey

As classes Predator e Prey são derivadas da classe Animal, com métodos para comer e reproduzir. Os Predadores comem as Presas e as Presas comem o alimento do terreno.

Descrição da Classe Hunter

A classe Hunter é um Animal, com métodos para comer/matar, reproduzir, morrer, ativar/desativar o sistema de partículas na morte e camuflar.

Descrição da Classe Player

A classe Player é um Animal, com métodos para consumir e obter energia, gerir as vidas e andar. Como dito anteriormente, o Player consegue matar todos os animais, mas no caso destes serem Leões ou Zebras, o Herói perde uma vida.

Descrição da Classe WorldConstants

A classe WorldConstants é uma classe com os valores constantes para a construção e funcionamento do Ecossistema.

Descrição da Classe Ecosystem

A classe Ecosystem é a classe que gere e “dá vida” ao ecossistema do jogo e tem métodos para mover os Animais consoante os seus papéis no ecossistema, colocar os animais a comer, remover os animais mortos e inicializar, adicionar e remover os diversos tipo de animais existentes no ecossistema, obter energia e morrer.

Descrição da Classe GameApp

A classe GameApp é classe onde se gere o jogo em si. Ao arrancar a app, temos um menu, com dois botões, um que leva a um Menu de Informações do jogo e outro que mostra os créditos do jogo. Ao estarmos no menu e clicarmos no ESPAÇO, fazemos com que o jogo comece. O jogo tem uma imagem da savana de fundo, do lado esquerdo tem-se o viewport onde o jogo aparece, emoldurado por outro viewport e do lado direito tem-se a barra de energia do Player, as vidas do mesmo; a contagem em tempo real de cada espécie no sistema (ao clicar na cabeça dos animais, surgem mais dos mesmos); um botão de restart, play (que ativa o jogo com o Player) e um botão de mute para o som de fundo.

Caso o Herói morra, surge um ecrã de Game Over.

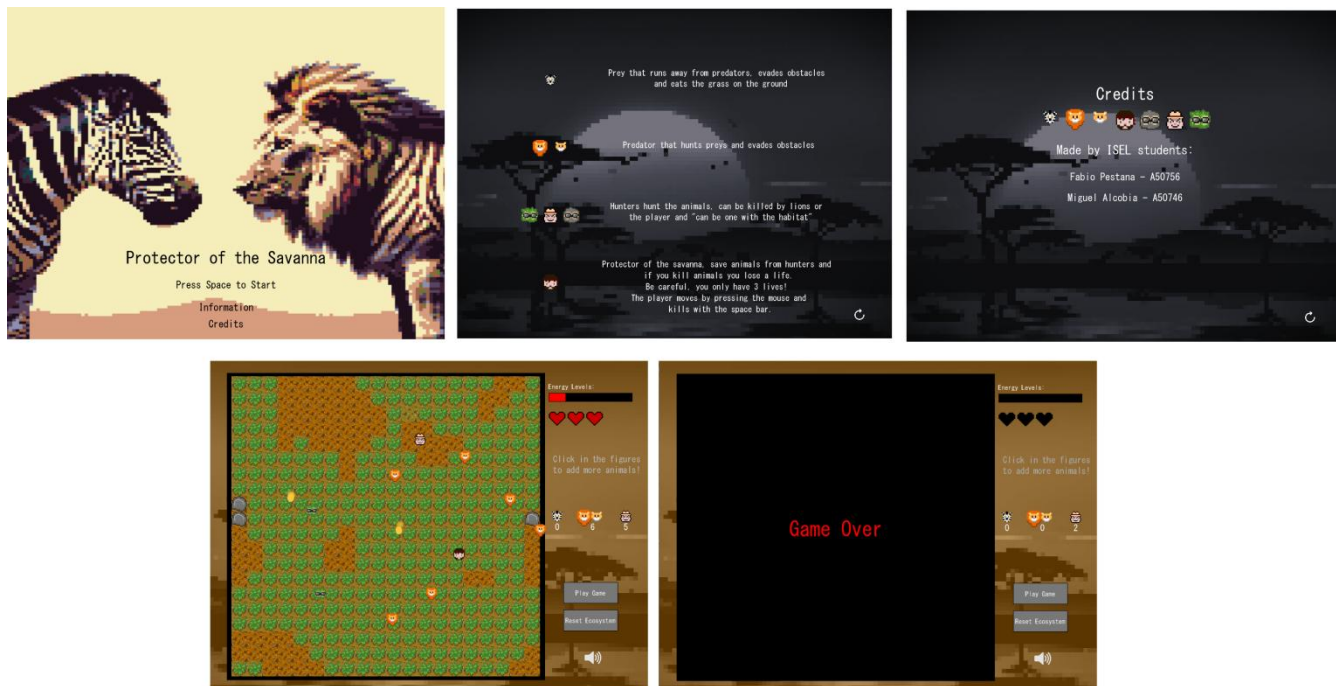


Figura 8 - Diversos Ecrãs do jogo

Conclusões

Com a realização deste projeto, percebemos que mesmo que a teoria seja importante, tem algumas divergências para com a realidade, como aconteceu entre o Silico e o jogo. A realidade tem bastantes variáveis que afetam os resultados obtidos de forma teórica. Também não foi possível equilibrar o sistema, devido aos valores usados para as constantes de gastos, obtenção de energia e reprodução dos animais.

Gostámos bastante de desenvolver o jogo e ir vendo o mesmo a evoluir ao longo projeto com diversas *features* que achámos interessante implementar.

Bibliografia e Ferramentas

Consulta:

Vieira, Paulo. (2023 - 2024). Física e Movimento - Moodle

Vídeos gravados pelo Prof. Arnaldo Abrantes - Moodle

Ferramentas:

Silico (Stocks & Flows:

Home. (n.d.). Silico.app. Retrieved January 28, 2024, from <https://silico.app/>

Pixel Art:

Descottes, J. (2019). *Piskel - Free online sprite editor*. Piskelapp.com. <https://www.piskelapp.com/>